

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score,
precision_score, recall_score, f1_score
from sklearn.linear_model import SGDClassifier
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt
```

Q1. Load the cleaned kidney dataset(ckd)dataset and print its type and description

```
df = pd.read_excel('C:/Users/Asus/Desktop/DL Lab/DL Lab/Assignment
3/ckd_clean1.xlsx')
print(df.dtypes)
print(df.describe())
```

Age	int64
Blood Pressure	int64
Specific Gravity	float64
Albumin	int64
Sugar	int64
Red Blood Cells	object
Pus Cell	object
Pus Cell clumps	object
Bacteria	object
Blood Glucose Random	int64
Blood Urea	int64
Serum Creatinine	float64
Sodium	int64
Potassium	float64
Hemoglobin	float64
Packed Cell Volume	int64
White Blood Cell Count	int64
Red Blood Cell Count	float64
Hypertension	object
Diabetes Mellitus	object
Coronary Artery Disease	object
Appetite	object
Pedal Edema	object
Anemia	object
Class	int64

dtype: object

	Age	Blood Pressure	Specific Gravity	Albumin
Sugar \				
count	158.000000	158.000000	158.000000	158.000000
mean	49.563291	74.050633	1.019873	0.797468
std	15.512244	11.175381	0.005499	1.413130

```

0.813397
min      6.000000      50.000000      1.005000      0.000000
0.000000
25%      39.250000      60.000000      1.020000      0.000000
0.000000
50%      50.500000      80.000000      1.020000      0.000000
0.000000
75%      60.000000      80.000000      1.025000      1.000000
0.000000
max      83.000000      110.000000      1.025000      4.000000
5.000000

```

	Blood Glucose Random	Blood Urea	Serum Creatinine	Sodium
\count	158.000000	158.000000	158.000000	158.000000
mean	131.341772	52.575949	2.188608	138.848101
std	64.939832	47.395382	3.077615	7.489421
min	70.000000	10.000000	0.400000	111.000000
25%	97.000000	26.000000	0.700000	135.000000
50%	115.500000	39.500000	1.100000	139.000000
75%	131.750000	49.750000	1.600000	144.000000
max	490.000000	309.000000	15.200000	150.000000

	Potassium	Hemoglobin	Packed Cell Volume	White Blood Cell
Count \count	158.000000	158.000000	158.000000	
mean	4.636709	13.687342	41.917722	
std	3.476351	2.882204	9.105164	
min	2.500000	3.100000	9.000000	
25%	3.700000	12.600000	37.500000	
50%	4.500000	14.250000	44.000000	
75%	4.900000	15.775000	48.000000	
max	47.000000	17.800000	54.000000	

	Red Blood Cell Count	Class
count	158.000000	158.000000
mean	4.891772	0.272152
std	1.019364	0.446483
min	2.100000	0.000000
25%	4.500000	0.000000
50%	4.950000	0.000000
75%	5.600000	1.000000
max	8.000000	1.000000

Q2. Prepare the data for Logistic Regression Take Logistic regression in the python library. Parameter tuning for optimizing classifier performance set for several parameter values namely: $C=1.0$, $\lambda=1.2$ C . The default value is 1.0. The smaller C value specifies the stronger regularization value. The Lambda parameter used to control the regularization strength. The increase in the lambda values the strength of the regularization effect.

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV
log_reg = LogisticRegression(max_iter=1000)
param_grid = {'C': [0.01, 0.1, 1.0, 10], 'penalty': ['l2']}
grid_search = GridSearchCV(log_reg, param_grid, scoring='accuracy',
cv=5)
grid_search.fit(X_train, y_train)
best_params = grid_search.best_params_
print("Best parameters: ", best_params)
categorical_cols = ['Red Blood Cells', 'Pus Cell', 'Pus Cell clumps',
'Bacteria', 'Hypertension', 'Diabetes Mellitus',
'Coronary Artery Disease', 'Appetite', 'Pedal
Edema', 'Anemia']
label_encoders = {}
for col in categorical_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col].astype(str))
    label_encoders[col] = le
X = df.drop('Class', axis=1)
y = df['Class']

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

Best parameters: {'C': 0.1, 'penalty': 'l2'}
```

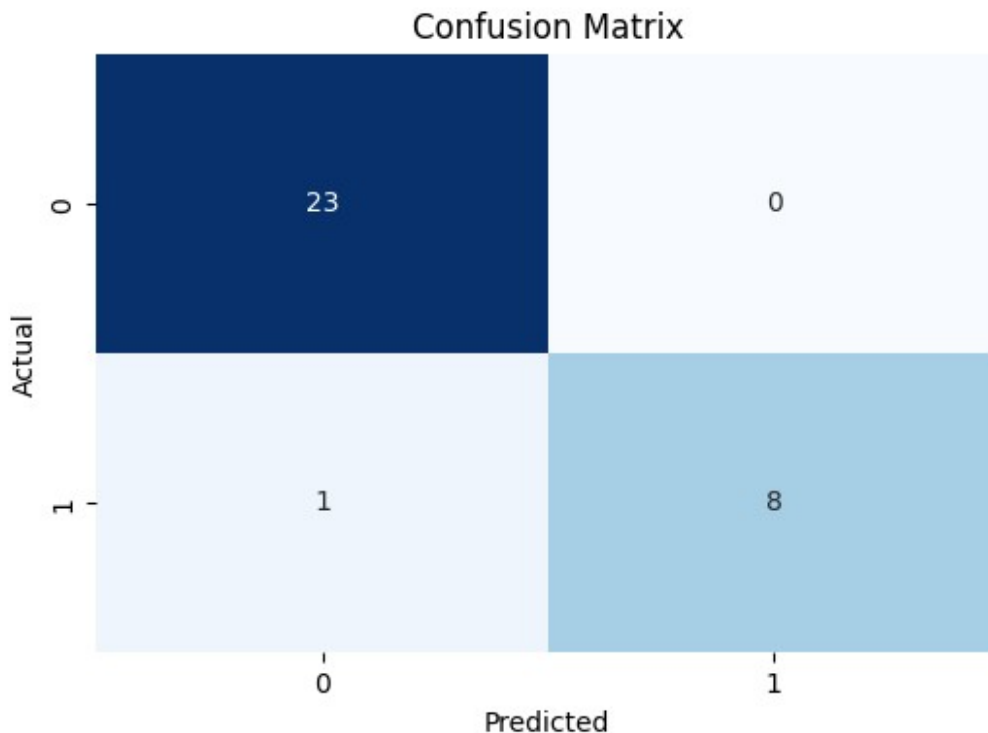
Logistic Regression without SGD

```
lr = LogisticRegression(max_iter=1000, C=1.0)
lr.fit(X_train, y_train)

LogisticRegression(max_iter=1000)
```

Q3. Confusion Matrix

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
log_reg = LogisticRegression(C=best_params['C'], penalty='l2',
max_iter=1000)
log_reg.fit(X_train, y_train)
y_pred = log_reg.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



Q4. Accuracy, Precision, F-measure, Recall

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f_measure = f1_score(y_test, y_pred)
print("Without SGD - Accuracy: {}, Precision: {}, Recall: {}, F-
measure: {}".format(accuracy, precision, recall, f_measure))
```

Without SGD - Accuracy: 0.96875, Precision: 1.0, Recall: 0.8888888888888888, F-measure: 0.9411764705882353

Q5. Logistic Regression with SGD

```
from sklearn.linear_model import SGDClassifier
sgd_clf = SGDClassifier(loss='log', alpha=0.0001, epsilon=0.1,
max_iter=1000, tol=1e
sgd_clf.fit(X_train, y_train)

Cell In[20], line 2
    sgd_clf = SGDClassifier(loss='log', alpha=0.0001, epsilon=0.1,
max_iter=1000, tol=1e
^
SyntaxError: invalid syntax
```

Q6. Confusion Matrix with SGD

```
y_pred_sgd = sgd_clf.predict(X_test)
cm_sgd = confusion_matrix(y_test, y_pred_sgd)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

-----
-----
NameError                                Traceback (most recent call
last)
Cell In[19], line 1
----> 1 y_pred_sgd = sgd_clf.predict(X_test)
      2 cm_sgd = confusion_matrix(y_test, y_pred_sgd)
      3 plt.figure(figsize=(6, 4))

NameError: name 'sgd_clf' is not defined
```

Q7. Accuracy, Precision, F-measure, Recall with SGD

```
accuracy_sgd = accuracy_score(y_test, y_pred_sgd)
precision_sgd = precision_score(y_test, y_pred_sgd)
recall_sgd = recall_score(y_test, y_pred_sgd)
f_measure_sgd = f1_score(y_test, y_pred_sgd)
print("With SGD - Accuracy: {}, Precision: {}, Recall: {}, F-measure:
{}".format(accuracy_sgd, precision_sgd, recall_sgd, f_measure_sgd))
```

With SGD - Accuracy: 0.28125, Precision: 0.28125, Recall: 1.0, F-measure: 0.43902439024390244

Q8. Plotting the results

```
labels = ['Accuracy', 'Precision', 'Recall', 'F-measure']
without_sgd = [accuracy, precision, recall, f_measure]
with_sgd = [accuracy_sgd, precision_sgd, recall_sgd, f_measure_sgd]
x = range(len(labels))
plt.plot(x, without_sgd, label='Without SGD', marker='o')
plt.plot(x, with_sgd, label='With SGD', marker='o')
plt.xticks(x, labels)
plt.ylabel('Score')
plt.title('Comparison of Logistic Regression and SGD')
plt.legend()
plt.show()
```

